

基于标量乘法的高效刷写式缓存攻击

周平¹、王涛¹、楼晓轩³、赵新杰²、张帆^{3,4*}、郭世泽²

¹中国石家庄市军械工程学院信息工程系, 邮编050003;

²中国北京电子设备北方研究所, 邮编100191;

³浙江大学信息科学与电气工程学院, 中国杭州 310027;

⁴中国成都610041 通信安全实验室

接收日期: 2017年2月20日/修订日期: 2017年5月9日/接受日期: 2017年5月19日/在线发表日期: 2017年8月16日

Citation Zhou P, Wang T, Lou X X, et al. 基于标量乘法签名算法的高效刷写重装缓存攻击. 《中国科学信息》, 2018, 61(3): 039102, doi: 10.1007/s11432-017-9108-3

尊敬的编辑:

缓存时序攻击是一种极具破坏力的侧信道攻击技术, 专门用于破解加密系统。近期, 一种新型缓存攻击技术“刷新-重载”被提出, 专门针对多核平台上的加密系统。该技术通过监控共享内存页的受攻击进程V, 利用间谍进程S对目标进程进行监控。S会刷新V的特定内存行, 清空各级缓存中的数据, 随后将这些内存行重新加载到缓存中。通过测量对应的加载时间, 攻击者就能精准追踪目标进程对特定指令的访问行为。

在刷写攻击中, 内存行监控位置的选择至关重要。文献[1]中, 基于平方乘法的RSA算法存在漏洞, 攻击者会锁定循环体中的内存行; 文献[2]则针对蒙哥马利梯形 ECDSA, 选择条件分支各分支中的内存行。然而当采用二进制方法实现标量乘法时, 现有方法均不适用于椭圆曲线(EC)签名算法(如 ECDSA 和 SM2-DSA [3], 见图1(a))。具体而言, 要精准定位这些内存行存在显著难度。

在循环体中频繁执行, 如[1]所示。在[2]中, 二进制方法的条件分支源代码非常简短, 导致不同分支对应的机器码可能位于同一或相邻的存储行中。因此, 间谍程序无法区分执行的是哪个分支。同时, 在恢复这些标量位时需要处理极高的错误率。

在本文中, 我们提出了一种新的内存行选择方法, 专门用于监控包含多个函数调用的内存行。这些函数调用会显著增加攻击者捕获受害者缓存访问概率的风险。我们将该方法应用于OpenSSL中基于EC的签名算法(采用二进制方法), 结果表明仅需一次签名即可以79%的概率完整恢复标量位。该方法同样适用于基于EC的滑动窗口签名和wNAF签名算法, 几乎能恢复所有倍增和加法操作, 错误率低至0.15%。

刷新-重载缓存攻击。在基于椭圆曲线的签名中, 二进制方法通过扫描标量k的比特位, 计算曲线上的点P, 其标量乘法式为 $P = [k]Q$, 其中Q为基点。攻击目标是恢复k。如图所示

*通讯作者(邮箱: fanzhang@zju.edu.cn) 作者声明无利益冲突。

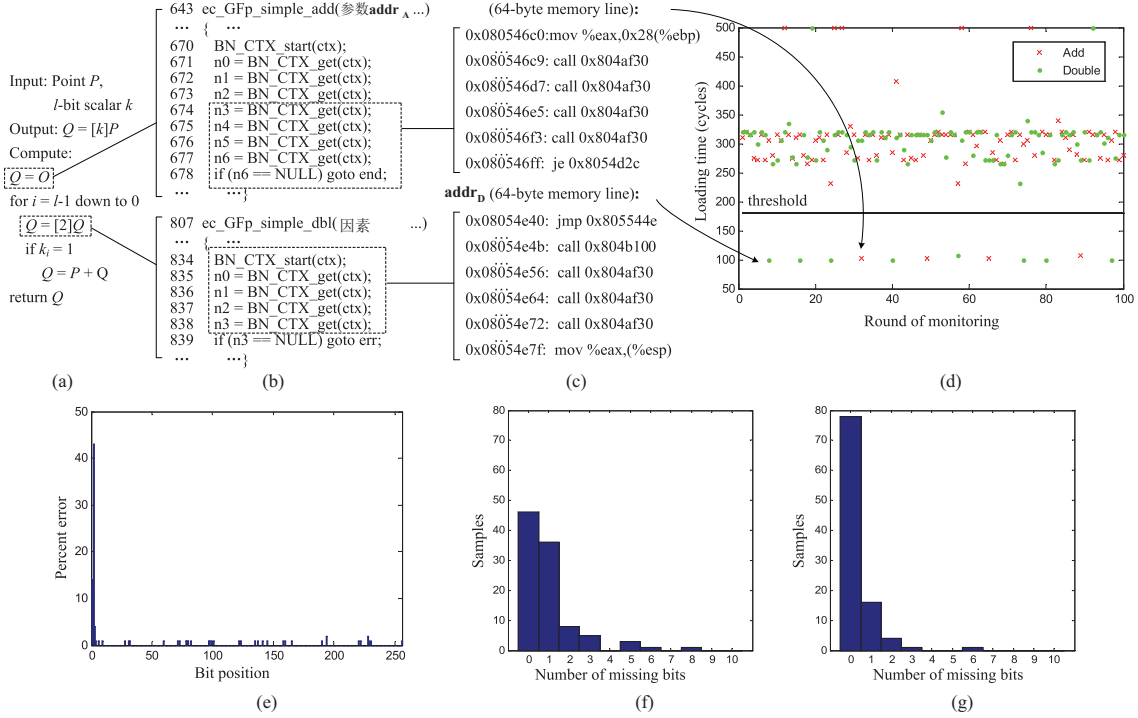


图1 (彩色在线) 监控内存行的选择与攻击结果。(a)二进制方法; (b)OpenSSL实现的加载时间累加与倍增; (c)监控内存行的位置; (d)加载时间的测量; (e)每比特位置的错误位数; (f)每标量的错误位数; (g)每标量的错误位数 (排除前2位)。

如图1(a)所示, 系统会先对每个 k_i 执行倍增操作, 若当前位被置为1则进行加法运算。在本文后续讨论中, 我们将倍增和加法分别记为D和A。为恢复AD序列并进一步提取标量位, Flush-Reload攻击会执行以下步骤。

阶段1: 为执行加法和倍增操作, 系统会选取指令地址的监控存储线, 分别标记为 addr_A 和 addr_D 。

阶段2: 通过计时监控缓存侧信道泄漏。每轮监控包含三个阶段。(1)在刷新阶段, S使用x86处理器架构支持的`clflush`指令, 将行地址 A 和行地址 D 从所有层级缓存中驱逐。(2)在触发阶段, S等待时间间隔 t_w 而不执行任何操作。在此阶段, V执行加法或倍增操作, 并触发从行地址 A 或 addr_D 加载相应内存。(3)在重载阶段, S重载内存行地址 A 和行地址 D , 并分别测量时间 t_A 和 t_D 。

S重复三个阶段直到达到签名的末尾。在每轮监控中, 会生成一个缓存访问状态向量, 记为 $t_i = [t_{iA}, t_{iD}]^T$ 。n轮监控的向量形成缓存访问轨迹 $T = [t_1, t_2, \dots, t_n]$, 如图1(d)所示。

阶段3: 每个 t_i 对应三种操作 o_i 中的一种: 倍增、加法或其余操作, 分别记为A、D、R。通过阈值 h 从 t_i 推导出 o_i , 得到序列 $O = [o_1, o_2, \dots, o_l]$:

$$o_i = \begin{cases} A, & \text{if } t_{iA} < h \wedge t_{iD} > h, \\ D, & \text{if } t_{iA} > h \wedge t_{iD} < h, \\ E, & \text{else.} \end{cases}$$

随后, 标量 k 可被推断为标量的比特值, 该值对应于相关操作。

我们的解决方案。为提高在群操作期间捕获受害者内存访问行为的概率, S需要监控频繁访问的内存行。然而在大多数标量乘法的实现中, 单次群操作的源代码通常非常简单直接。这里存在两个主要缺陷: 首先需要监控的内存行数量极少; 其次每条指令仅执行一次。因此当 t_w 较小时, S将难以准确捕捉正确的缓存行为。

然而, 攻击者可深入机器码层面, 利用函数调用机制实现对同一内存行的多次访问。当执行调用指令的机器码时, 进程会跳转至被调用函数的地址。执行后

在被调用函数中, 程序会返回并从调用指令后的地址继续执行。因此, 在单次执行过程中, 包含 n 条调用指令的内存行会被访问 $n+1$ 次。这种现象为攻击者提供了双重优势: 函数调用引发的多次访问增加了S捕获缓存行为的概率。此外, 被调用函数的执行将这些访问分散在不同的时间槽中, 使得这些访问更容易被检测和区分。

在我们构建的OpenSSL中, 我们选择了监控内存行 addr_A 和 addr_D , 其中源代码中负责加法运算的第674至678行以及负责乘法运算的第834至838行位于图1(b)所示位置。这些内存行包含最多的调用指令。以 addr_A 行为例, 其对应的机器码及其虚拟地址如图1(c)所示。该行包含四个调用指令, 单次执行时会导致对内存行的五次访问。因此, 即使存在潜在的重叠或无法区分的访问, S捕获至少一次受害者访问的概率也会显著增加。在极高的概率下, AD序列可以被完全恢复, 密钥也能被成功提取。

实验。实验在ThinkCentre台式机上进行, 该设备配备Intel i5-3470处理器、4 GB DDR3-1600内存及Fedora 18系统。标量乘法采用二进制方法实现, 加法与倍增操作通过OpenSSL 1.01e完成。256位标量和曲线参数均随机生成。我们将时间槽 t_w 设置为600个周期, 阈值 h 设为180个周期。

我们执行100次签名操作, 记录监控内存行的时间数据, 并尝试恢复标量值。平均总错误率为0.38% (256位标量中的0.96位), 低于文献[2]中4.26%的错误率。

这些错误位的位置分布如图1(e)所示。错误主要集中在前两位, 这可能是由于执行初期的稳定性不足所致。其余位的错误率均低于2%。图1(f)展示了100个签名中错误位数量的分布情况。所有恢复的标量数据中, 错误位数均不超过8位, 其中46个完全正确。由于错误主要集中在前两位(其数值可通过简单的暴力破解法轻松验证), 在标量恢复过程中可忽略这些错误。因此,

100个签名中错误位数的分布如图1(g)所示。所有恢复的标量均不超过6个错误位, 其中79个完全正确。

结论。本文提出了一种新方法, 用于筛选因函数调用而被多次访问的监控内存行。实验表明, 当签名采用标量乘法二进制算法时, 我们改进的Flush-Reload攻击方法可实现79%的概率恢复密钥。该研究还可拓展至滑动窗口法和wNAF法的标量乘法攻击。对于256位标量和3位窗口, AD序列的错误率低至0.15%, 且在73%的签名中能完全恢复AD序列。由于滑动窗口法或wNAF法无法从AD序列中完全恢复标量, 本攻击需结合格技术[4, 5]。攻击者需要约344个签名才能恢复私钥。

致谢 本研究部分由国家基础研究计划(973计划)(项目编号: 2013CB338004)和国家自然科学基金(项目编号: 61272491、61309021、61472357、61571063)资助。

支持信息附录A—C。支持信息可在info.scichina.com和link.springer.com在线获取。支持材料以提交形式出版, 未经排版或编辑。科学准确性及内容的责任完全由作者承担。

参考文献

- 1 Yarom Y, Falkner K. Flush+reload: 一种高分辨率、低噪声的L3缓存侧信道攻击。载于: 第23届USENIX安全研讨会论文集, 圣地亚哥, 2014年。719—732
- 2 Yarom Y, Benger N. 利用flush+reload缓存侧信道攻击恢复OpenSSL ECDSA 密钥。IACR 密码学电子预印本档案, 2014年, 第140页
- 3 中国国家密码管理局。基于椭圆曲线的公钥密码算法SM2。GM/T 0003-2012。http://www.oscca.gov.cn/UpFile/2010122214822692.pdf
- 4 Nguyen P Q, Shparlinski I E. 部分已知密钥椭圆曲线数字签名算法的不安全性。设计密码学代码, 2003, 30: 151—176
- 5 刘敏、陈杰。SM2签名算法的部分已知非对称性和故障注入攻击。《信息安全与密码学会会议论文集》, 广州, 2013。343—358。